Documentación de la API

Actividad individual, módulo 2. Arquitectura Web, Universidad de Palermo, 2do cuatrimestre 2026.
Alumno: Juan Manuel Gonzalez Kapnik · Profesor: Ing. Diego Marafetti

---

1. Descripción general del backend

El backend expone una **API REST para administrar un catálogo personal de películas y series**.

El dominio que resuelve es la **gestión de una lista de títulos por ver y ya vistos**. El problema es el de cualquiera que acumula pendientes repartidos entre varias plataformas y termina sin saber qué tiene, cuánto le falta ni cuánto tiempo le llevaría. La API centraliza el catálogo y responde esas preguntas.

Las operaciones que ofrece son:

- **CRUD completo de títulos**: alta, consulta individual, listado, reemplazo total, modificación parcial y baja.
- **Consulta filtrada del catálogo** por tipo, género, director, idioma, estado de visionado, puntaje mínimo, rango de años y búsqueda por texto, con orden y paginado.
- **Consulta de géneros y directores**, derivados de los títulos cargados.
- **Dos reportes**: estadísticas de visionado y puntaje promedio por género.

El recurso central es el **título**, que puede ser una **película** (`movie`) o una **serie** (`series`). La distinción sostiene las reglas de negocio: una serie declara temporadas y episodios, una película no, y el cálculo de horas de un reporte los trata distinto.

Los datos iniciales son **45 títulos (35 películas y 10 series)**, con 16 géneros y 31 directores, cargados automáticamente al iniciar el servidor.

La implementación es Node.js con Express, con persistencia en memoria detrás de un repositorio. El diseño corresponde al **nivel 2 del modelo de madurez de Richardson**: cada recurso tiene su URL, el verbo indica la operación y el status code indica el resultado.

---

2. Convenciones generales

URL base: `http://localhost:3000` **Prefijo de la API**: `/api/v1` **Formato**: JSON (`application/json`) en pedidos y respuestas.

> **Estructura de las respuestas**

Las respuestas exitosas devuelven el resultado dentro de `data` . Los listados agregan `meta` con la información de paginado.

```
{
  "data": [ { "id": 5, "title": "Inception", "type": "movie", "year": 2010 } ],
  "meta": { "total": 45, "page": 1, "limit": 20, "pages": 3 }
}
```

Todos los errores comparten el mismo formato:

```
{
  "error": {
    "status": 422,
    "code": "UNPROCESSABLE_ENTITY",
    "message": "El titulo no cumple las reglas del catalogo.",
    "details": [ { "field": "seasons", "message": "una pelicula no tiene temporadas" } ]
  }
}
```

El campo `details` aparece solo cuando hay información por campo.

Respuestas comunes a todos los endpoints

Status	Cuándo ocurre
`404 Not Found`	La ruta pedida no existe en la API
`405 Method Not Allowed`	La ruta existe pero no acepta ese verbo. La respuesta incluye el header `Allow` con los verbos aceptados
`413 Payload Too Large`	El cuerpo del pedido supera los 100 kB (solo en endpoints con body)
`500 Internal Server Error`	Error inesperado del servidor

Orden en que se validan los pedidos

Cuando un pedido tiene más de un problema, el status code que se devuelve es el del primer control que falla. El orden es:

1. **`400`** si el cuerpo no es JSON válido, o **`413`** si supera el tamaño máximo.
2. **`400`** si el `{id}` de la URL no es un número entero.
3. **`415`** si el `Content-Type` no es `application/json`.
4. **`404`** si el título indicado no existe.
5. **`400`** si algún campo del cuerpo es inválido, falta, o es desconocido.
6. **`409`** si ya existe otro título con el mismo nombre y año.
7. **`422`** si el cuerpo viola una regla de negocio.

---

3. El recurso Título

Campo	Tipo	Obligatorio	Reglas
`id`	entero	lo asigna el servidor	Enviarlo en el body devuelve `400`
`title`	texto	sí	Entre 1 y 200 caracteres. Junto con `year` forma la identidad del título y no se puede repetir
`type`	texto	sí	Solo `movie` o `series`
`director`	texto	sí	Entre 2 y 120 caracteres
`year`	entero	sí	Entre 1888 (el primer largometraje conocido) y el año actual más 5, para admitir estrenos ya anunciados
`genres`	lista de texto	sí	Entre 1 y 5 géneros, sin repetidos, hasta 40 caracteres cada uno
`runtimeMinutes`	entero	sí	Entre 1 y 1000. En una serie representa la **duración promedio de un episodio**
`seasons`	entero o `null`	no	Entre 1 y 100. Obligatorio si `type` es `series`, prohibido si es `movie`. Por defecto `null`
`episodes`	entero o `null`	no	Entre 1 y 10000. Obligatorio si `type` es `series`, prohibido si es `movie`. Por defecto `null`
`language`	texto	no	Código ISO de dos letras minúsculas (`es`, `en`, `ja`). Por defecto `en`
`watched`	booleano	no	Por defecto `false`
`rating`	entero o `null`	no	Entre 1 y 10. Solo se admite si `watched` es `true`. Por defecto `null`
`watchedOn`	texto o `null`	no	Fecha `AAAA-MM-DD` existente y no futura. Solo se admite si `watched` es `true`. Por defecto `null`

Reglas de negocio

Estas reglas se verifican después de la validación de forma y devuelven `422`:

1. Un título de tipo `series` tiene que declarar `seasons` y `episodes`.
2. Un título de tipo `movie` no puede declarar `seasons` ni `episodes`.
3. No se puede puntuar (`rating`) un título que no está marcado como visto.
4. No se puede fechar (`watchedOn`) un título que no está marcado como visto.
5. `watchedOn` no puede ser una fecha futura.

Ejemplos de un título de cada tipo:

```
{
  "id": 5,
  "title": "Inception",
  "type": "movie",
  "director": "Christopher Nolan",
  "year": 2010,
  "genres": ["Action", "Sci-Fi", "Thriller"],
  "runtimeMinutes": 148,
  "seasons": null,
  "episodes": null,
  "language": "en",
  "watched": true,
  "rating": 9,
  "watchedOn": "2024-05-11"
}
```

```
{
  "id": 36,
  "title": "Breaking Bad",
  "type": "series",
  "director": "Vince Gilligan",
  "year": 2008,
  "genres": ["Crime", "Drama", "Thriller"],
  "runtimeMinutes": 49,
  "seasons": 5,
  "episodes": 62,
  "language": "en",
  "watched": true,
  "rating": 10,
  "watchedOn": "2025-02-22"
}
```


---

4. Endpoints

4.1. `GET /`

Propósito: índice de la API. Devuelve el nombre, la versión y la lista de rutas disponibles, para que un cliente pueda descubrir el resto sin leer la documentación.

Status	Cuándo ocurre
`200 OK`	Siempre
`405 Method Not Allowed`	Cualquier verbo que no sea `GET`. Header `Allow: GET`

4.2. `GET /api/v1/health`

Propósito: verificar que el servicio está vivo. Informa hace cuántos segundos está corriendo y cuántos títulos cargó el seed.

```
{ "status": "ok", "uptimeSeconds": 12, "seededTitles": 45 }
```

Status	Cuándo ocurre
`200 OK`	El servicio responde
`405 Method Not Allowed`	Cualquier verbo que no sea `GET`. Header `Allow: GET`

4.3. `GET /api/v1/titles`

Propósito: listar los títulos del catálogo. Acepta filtros, orden y paginado, todos opcionales y combinables entre sí.

Parámetros de consulta (query string):

Parámetro	Valores	Por defecto	Qué hace
`type`	`movie` / `series`	—	Filtra por tipo de título
`genre`	texto	—	Filtra por un género, sin distinguir mayúsculas
`director`	texto	—	Filtra por coincidencia parcial en el director
`language`	texto	—	Filtra por código de idioma
`watched`	`true` / `false`	—	Filtra por vistos o pendientes
`minRating`	entero de 1 a 10	—	Deja solo los títulos con puntaje mayor o igual. Los no puntuados quedan afuera
`yearFrom`	entero ≥ 1888	—	Año de estreno mínimo
`yearTo`	entero ≥ 1888	—	Año de estreno máximo
`q`	texto, mínimo 2 caracteres	—	Busca el texto en el título o en el director
`sort`	`id`, `title`, `year`, `rating`, `runtimeMinutes`	`id`	Campo por el que se ordena
`order`	`asc` / `desc`	`asc`	Sentido del orden
`page`	entero ≥ 1	`1`	Página a devolver
`limit`	entero entre 1 y 100	`20`	Cantidad de títulos por página

Al ordenar por `rating`, los títulos sin puntaje quedan siempre al final, sea cual sea el sentido del orden.

Ejemplo: `GET /api/v1/titles?watched=false&sort=runtimeMinutes&limit=5`

Status	Cuándo ocurre
`200 OK`	El listado se resolvió, aun si ningún título coincide (en ese caso `data` es un arreglo vacío y `meta.total` es 0)
`400 Bad Request`	Un parámetro tiene un valor inválido: `type` distinto de `movie` / `series`, `sort` con un campo que no existe, `order` distinto de `asc` / `desc`, `page`, `limit`, `minRating`, `yearFrom` o `yearTo` no enteros o fuera de rango, `watched` distinto de `true` / `false`, o `q` con menos de 2 caracteres
`405 Method Not Allowed`	Verbos distintos de `GET` y `POST`. Header `Allow: GET, POST`

4.4. `POST /api/v1/titles`

Propósito: agregar un título al catálogo. El servidor asigna el `id` y devuelve el recurso creado junto con su ubicación en el header `Location`.

Cuerpo esperado (los campos opcionales se pueden omitir y toman su valor por defecto):

```
{
  "title": "Perfect Days",
  "type": "movie",
  "director": "Wim Wenders",
  "year": 2023,
  "genres": ["Drama"],
  "runtimeMinutes": 124,
  "language": "ja"
}
```

Status	Cuándo ocurre
`201 Created`	El título se creó. Devuelve el recurso completo y el header `Location` con su URL
`400 Bad Request`	El JSON está mal formado, falta un campo obligatorio, un tipo no corresponde, `type` está fuera del enum, `genres` viene vacío o con repetidos, la fecha no existe en el calendario, hay un campo desconocido, o se envió `id` en el cuerpo
`405 Method Not Allowed`	Verbos distintos de `GET` y `POST`. Header `Allow: GET, POST`
`409 Conflict`	Ya existe otro título con el mismo `title` y el mismo `year`
`413 Payload Too Large`	El cuerpo supera los 100 kB
`415 Unsupported Media Type`	El `Content-Type` no es `application/json`
`422 Unprocessable Entity`	El cuerpo es válido en su forma pero viola una regla de negocio: una película con `seasons` o `episodes`, una serie sin ellos, un `rating` o un `watchedOn` sobre un título no visto, o una fecha de visionado futura

No es idempotente: dos pedidos idénticos crearían dos títulos, salvo que el par `title` + `year` repetido lo impida con un `409`.

4.5. `GET /api/v1/titles/{id}`

Propósito: obtener un título puntual por su identificador.

Status	Cuándo ocurre
`200 OK`	El título existe. Devuelve el recurso completo
`400 Bad Request`	El `{id}` de la URL no es un número entero (ej. `/titles/abc`)
`404 Not Found`	No existe ningún título con ese id
`405 Method Not Allowed`	Verbos distintos de `GET`, `PUT`, `PATCH` y `DELETE`. Header `Allow: GET, PUT, PATCH, DELETE`

4.6. `PUT /api/v1/titles/{id}`

Propósito: reemplazar un título completo. Hay que enviar el recurso entero: los campos opcionales que se omitan **vuelven a su valor por defecto**, no conservan el valor anterior. Para cambiar solo algunos campos, usar `PATCH`.

Cuerpo esperado: el mismo del `POST`, sin el `id` (viaja en la URL).

Status	Cuándo ocurre
`200 OK`	El título se reemplazó. Devuelve el recurso actualizado
`400 Bad Request`	El `{id}` no es un entero, el JSON está mal formado, falta un campo obligatorio, un tipo no corresponde, hay un campo desconocido, o se envió `id` en el cuerpo
`404 Not Found`	No existe ningún título con ese id. **No crea el recurso**: los identificadores los asigna el servidor
`405 Method Not Allowed`	Verbos distintos de `GET`, `PUT`, `PATCH` y `DELETE`. Header `Allow: GET, PUT, PATCH, DELETE`
`409 Conflict`	El `title` y `year` enviados ya los usa otro título
`413 Payload Too Large`	El cuerpo supera los 100 kB
`415 Unsupported Media Type`	El `Content-Type` no es `application/json`
`422 Unprocessable Entity`	Alguna de las cinco reglas de negocio no se cumple

Es idempotente: repetir el mismo pedido deja el recurso en el mismo estado.

4.7. `PATCH /api/v1/titles/{id}`

Propósito: modificar parcialmente un título. Solo viajan los campos que cambian; el resto queda como estaba. Es el verbo natural para el uso más común del catálogo: marcar algo como visto y puntuarlo.

Cuerpo esperado: un objeto con al menos un campo del recurso.

```
{ "watched": true, "rating": 9, "watchedOn": "2026-08-17" }
```

Status	Cuándo ocurre
`200 OK`	El título se modificó. Devuelve el recurso actualizado completo
`400 Bad Request`	El `{id}` no es un entero, el JSON está mal formado, el cuerpo está vacío, un tipo no corresponde, hay un campo desconocido, o se envió `id` en el cuerpo
`404 Not Found`	No existe ningún título con ese id
`405 Method Not Allowed`	Verbos distintos de `GET`, `PUT`, `PATCH` y `DELETE`. Header `Allow: GET, PUT, PATCH, DELETE`
`409 Conflict`	El `title` y `year` resultantes ya los usa otro título
`413 Payload Too Large`	El cuerpo supera los 100 kB
`415 Unsupported Media Type`	El `Content-Type` no es `application/json`
`422 Unprocessable Entity`	El resultado de aplicar el cambio viola una regla de negocio. Por ejemplo, marcar como no visto un título que tiene puntaje cargado

---

4.8. `DELETE /api/v1/titles/{id}`

Propósito: eliminar un título del catálogo.

Status	Cuándo ocurre
`204 No Content`	El título se eliminó. La respuesta no tiene cuerpo
`400 Bad Request`	El `{id}` de la URL no es un número entero
`404 Not Found`	No existe ningún título con ese id, incluido el caso de repetir un `DELETE` ya efectuado
`405 Method Not Allowed`	Verbos distintos de `GET`, `PUT`, `PATCH` y `DELETE`. Header `Allow: GET, PUT, PATCH, DELETE`

Es idempotente, aunque el segundo pedido devuelva `404`. La idempotencia se define sobre el estado del sistema, no sobre el status code: después del primer `DELETE` el título ya no está, y repetirlo no cambia nada.

4.9. `GET /api/v1/genres`

Propósito: listar los géneros presentes en el catálogo con la cantidad de títulos de cada uno, de mayor a menor. Se derivan de los títulos cargados, por eso el recurso es de solo lectura.

```
{ "data": [ { "name": "Drama", "titleCount": 33 } ] }
```

Como un título puede tener hasta 5 géneros, la suma de los conteos es mayor que la cantidad de títulos.

Status	Cuándo ocurre
`200 OK`	Siempre
`405 Method Not Allowed`	Cualquier verbo que no sea `GET`. Header `Allow: GET`

4.10. `GET /api/v1/directors`

Propósito: listar los directores presentes en el catálogo con la cantidad de títulos de cada uno, de mayor a menor. También se derivan de los títulos.

```
{ "data": [ { "name": "Christopher Nolan", "titleCount": 3 } ] }
```

Status	Cuándo ocurre
`200 OK`	Siempre
`405 Method Not Allowed`	Cualquier verbo que no sea `GET`. Header `Allow: GET`

4.11. `GET /api/v1/reports/watch-stats`

Propósito: reporte de estadísticas de visionado. Responde cuánto se vio, cuánto falta, cuántas horas llevó y cuántas quedan, con el desglose entre películas y series. La lista de pendientes viene ordenada de menor a mayor duración, que es por donde conviene empezar.

Las horas de una serie se calculan como la duración promedio del episodio por la cantidad de episodios: sin esa distinción, una serie de seis temporadas "duraría" 49 minutos y el reporte no diría nada.

```
{
  "data": {
    "summary": {
      "totalTitles": 45,
      "watched": 35,
      "pending": 10,
      "completionPct": 77.78,
      "watchedHours": 235.9,
      "pendingHours": 180.58,
      "averageRating": 8.71
    },
    "byType": [
      { "type": "movie", "total": 35, "watched": 28, "pending": 7, "watchedHours": 63.18 },
      { "type": "series", "total": 10, "watched": 7, "pending": 3, "watchedHours": 172.72 }
    ],
    "pending": [ { "id": 11, "title": "Taxi Driver", "type": "movie", "year": 1976, "totalMinutes": 114 } ]
  }
}
```


Status	Cuándo ocurre
`200 OK`	Siempre
`405 Method Not Allowed`	Cualquier verbo que no sea `GET`. Header `Allow: GET`

4.12. `GET /api/v1/reports/rating-by-genre`

Propósito: reporte de puntaje promedio por género, de mayor a menor. Solo entran los títulos puntuados, porque un pendiente no tiene nota, y por eso cada género informa dos conteos distintos: cuántos títulos tiene y cuántos de ellos están puntuados.

Un título con varios géneros suma en todos, así que `genreAssignments` es mayor que `totalTitles`. El campo está explícito en el resumen justamente para que ese número no se lea como un error.


```
{
  "data": {
    "summary": { "totalTitles": 45, "genres": 16, "genreAssignments": 115 },
    "genres": [
      { "genre": "History", "titleCount": 2, "ratedCount": 1, "averageRating": 10 },
      { "genre": "Crime", "titleCount": 17, "ratedCount": 10, "averageRating": 9.3 }
    ]
  }
}
```


Un género cuyos títulos estén todos pendientes devuelve `averageRating: null` y se ordena al final.

Status	Cuándo ocurre
`200 OK`	Siempre
`405 Method Not Allowed`	Cualquier verbo que no sea `GET`. Header `Allow: GET`

---

5. Tabla resumen

Verbo	Ruta	Propósito	Status codes posibles
`GET`	`/`	Índice de la API	200, 405
`GET`	`/api/v1/health`	Estado del servicio	200, 405
`GET`	`/api/v1/titles`	Listar títulos	200, 400, 405
`POST`	`/api/v1/titles`	Crear un título	201, 400, 405, 409, 413, 415, 422
`GET`	`/api/v1/titles/{id}`	Obtener un título	200, 400, 404, 405
`PUT`	`/api/v1/titles/{id}`	Reemplazar un título	200, 400, 404, 405, 409, 413, 415, 422
`PATCH`	`/api/v1/titles/{id}`	Modificar parcialmente	200, 400, 404, 405, 409, 413, 415, 422
`DELETE`	`/api/v1/titles/{id}`	Eliminar un título	204, 400, 404, 405
`GET`	`/api/v1/genres`	Listar géneros	200, 405
`GET`	`/api/v1/directors`	Listar directores	200, 405
`GET`	`/api/v1/reports/watch-stats`	Estadísticas de visionado	200, 405
`GET`	`/api/v1/reports/rating-by-genre`	Puntaje promedio por género	200, 405

Cualquier ruta no listada devuelve `404 Not Found`, y cualquier error inesperado devuelve `500 Internal Server Error`.